

The Missing Software

Five parallel research tracks into what the software industry has not built: **unbuilt tools, broken plumbing, missing links, badly-built categories**, and the open gaps of the AI era. Nearly forty validated gaps, screened against incumbents, ranked by what a small team can actually ship.

SVX Research

Multi-Agent Research Series · Tracks R1-R5 · Edition 2

Table of Contents

1. Executive Summary	1
2. Methodology: Five Parallel Research Tracks	2
3. The Gap Map: Five Lenses on What Is Missing	3
4. Developer Tooling: Seven Validated Gaps	5
5. The Broken Plumbing: Integration, Data and Exit	8
6. Missing Links: Connective Tissue Never Built	11
7. Badly-Built Categories: Why Hated Software Stays Bad	15
8. Frontier Gaps: AI Infrastructure and Unsexy Industries	17
9. Cross-Cutting Patterns: Why These Gaps Persist	20
10. The Build Shortlist: What a Small Team Could Ship	21
Appendix A: Source Register	23

1. Executive Summary

This report asks a single question at the level of the whole industry: **what is the software industry missing?** Not "which product could be incrementally better," but which tools, connective links, and whole categories have never been built, or have been built so badly that they function as open wounds. The research deliberately prioritized gaps where nobody has done the work, or where everyone has done it badly, over crowded markets with strong incumbents. Everything that follows is filtered through that lens.

Five parallel research tracks were run, each covering a different angle of the industry: developer tooling; integration, data, and system plumbing; unbuilt "missing links" between ecosystems; long-hated but never-fixed software categories; and frontier gaps spanning AI-era infrastructure and unsexy industries. Together they executed roughly sixty web searches, harvested evidence predominantly from 2024 through 2026, and discarded every candidate gap that turned out to be already served by a strong solution. The survivors are presented here with their evidence, their structural causes, and a realistic angle for a small team to ship them.

96%

of developers do not fully trust AI-generated code, yet only 48% always verify it (SonarSource, Jan 2026)

\$315K

average vendor lock-in cost per enterprise migration, with 62% citing data migration as the failure driver (Kong, Jun 2026)

79%

of opportunity data that reps collect never makes it into the CRM (DevRev, 2026)

Three headline conclusions

- **The industry has a translation problem, and AI just changed the economics of it.** Nearly every durable gap found — design to code, spec to test, docs to runtime, prose to formal properties — stayed unbuilt because translating between two representations cost more than the failures it prevented. That translation cost collapsed in 2025-2026, which is why so many decades-old gaps suddenly became buildable.
- **The deepest gaps are ownership vacuums, not technology vacuums.** Integrations break silently because nobody is paid to watch them; documentation rots because no test suite goes red; permission graphs drift because identity syncs accounts, not access. Connectivity exists almost everywhere — ownership does not.
- **Buyer-user mismatch is the one pattern AI does not fix.** In hated categories (CRM entry, expense reports, timesheets), the person who suffers is not the person who buys. The viable wedge is bottom-up: sell the suffering user a tool that removes their work, then sell the clean exhaust data to the buyer.

The strongest immediately buildable opportunities for a two-to-four person team are: an **evals-in-CI adapter** that makes non-deterministic AI systems testable inside existing pipelines; a **verification and provenance ledger for AI-generated code**, sitting at the merge gate where trust currently evaporates; and an **integration observability proxy** that watches the brittle wiring between SaaS systems that nobody else monitors. Chapter 10 ranks twelve such opportunities by evidence strength, openness of the field, and realism of the distribution model.

2. Methodology: Five Parallel Research Tracks

The research was structured as five independent tracks, each assigned to a separate research agent so that no single perspective would dominate the findings. Track R1 examined developer tooling: environments, testing, observability, documentation, code review, and dependency management. Track R2 examined the plumbing layer: B2B integration, the small-business data layer, migration and exit tooling, identity and permissions, legacy modernization, and batch-file interchange. Track R3 hunted specifically for "missing links" — pairs of systems that both exist and both work but were never connected. Track R4 analyzed badly-built categories: software used for ten to twenty years that nearly everyone hates, and the structural reasons it stays bad. Track R5 covered the two frontiers: missing infrastructure for the AI application era, and unsexy industries still running on spreadsheets.

Each track applied the same verification rule, which is the methodological core of this report: **every candidate gap was checked against the strongest incumbent solution before being accepted.** Docker and Nix exist, so "dev environments are unsolved" was rejected and narrowed to the residual slice that remains — composition, secrets, seed data, and verification. The API contract-testing ecosystem is mature and growing (a reported \$2.8B services market), so it was discarded as a gap. LLM tracing was discarded because nine or more mature tools already compete. This screening is what separates the findings from a generic list of startup ideas: everything in this report survived a deliberate attempt to kill it.

Evidence standards required that every claimed pain point trace to a real, dated source — a survey, a vendor report, an engineering post, or a practitioner discussion — seen directly in search results during the research window. No statistics were invented or extrapolated. The evidence window prioritizes 2024-2026, with older statistics included only where explicitly flagged. Two limitations deserve honesty: shared search-quota rate limits forced some tracks to lean on cached result sets rather than fresh verification searches, and a handful of unsexy-industry verticals (field services, small-manufacturing MES, elder care, government legacy) are flagged in Chapter 8 as recommended follow-up research rather than asserted as validated gaps.

3. The Gap Map: Five Lenses on What Is Missing

Across the five tracks, the validated gaps sort into a surprisingly clean map. Each lens asks a different question about the industry, and each produced six to ten gaps that survived verification. The table below is the index to the rest of the report: every chapter that follows elaborates one row of this map with full evidence and buildable angles.

Lens	Core question	Strongest validated gaps
Developer tooling (Ch. 4)	What does the daily workflow still lack?	Flaky-test root-cause repair; flat-priced observability for small teams; docs that go red when stale; review scaled to AI-code volume; dependency-update judgment
Plumbing: integration and data (Ch. 5)	What is broken underneath working systems?	Integration debt with no owner; the missing SMB data layer; exit and migration tooling; permission-graph drift; spreadsheet-as-database
Missing links (Ch. 6)	What exists on both ends but was never connected?	Design intent to production drift CI; spec-to-test trace linter; self-verifying docs; AI-code verification ledger; deploy-to-incident causal join
Badly-built categories (Ch. 7)	Why does hated software stay hated?	CRM as entry system; the expense-report dead zone; meeting notes that never execute; email-client graveyard economics
Frontier gaps (Ch. 8)	What does the new era lack underneath it?	Evals-in-CI for non-deterministic AI; portable agent memory; code provenance attestation; AI FinOps allocation; construction sub-tier estimating

Table 1. The five research lenses and their strongest validated gaps

The gaps are not abstract. Practitioners quantify them constantly, in surveys and incident reports that the industry then fails to productize. The chart below collects six of the most telling quantifications found during the research — a trust gap inside the merge gate, a data-entry gap that hollows out the system of record, and an estimating practice that keeps an entire trade vertical in spreadsheet land. Each number corresponds to a validated gap later in the report, and each one is a survey result, not an estimate.

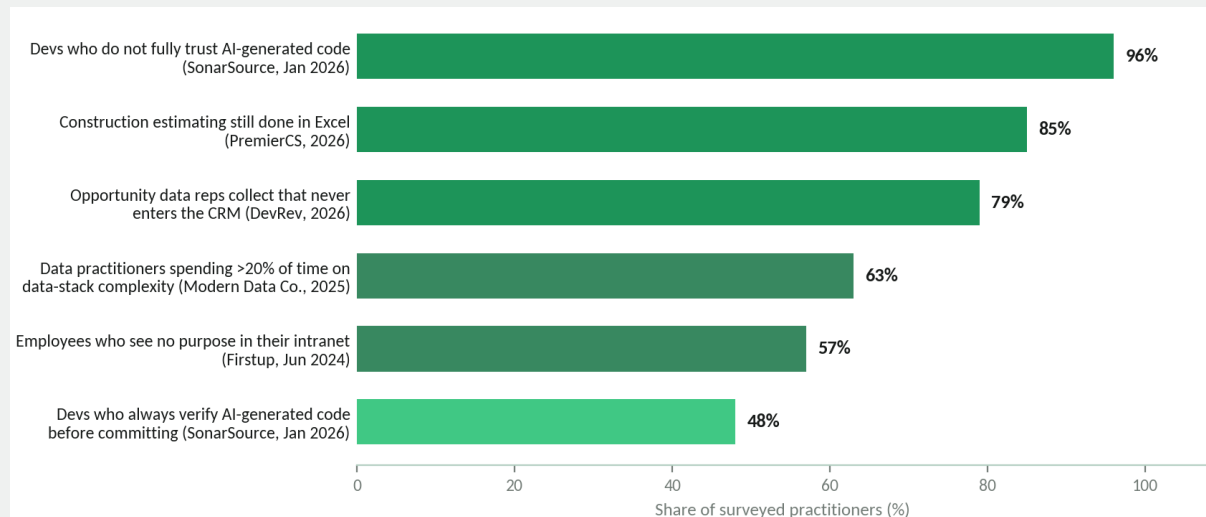


Figure 1. Quantified pain points across the validated gaps (share of surveyed practitioners; sources as labeled)

Reading the six numbers together reveals the shape of the opportunity. The two SonarSource figures form a single behavioral scandal: almost nobody trusts machine-written code, and barely half of the profession verifies it — which is precisely the gap a verification layer can monetize. The 79% CRM figure means the "system of record" is structurally blind to most of the reality it is supposed to record. And the 85% Excel figure in construction is not a laggard statistic; it is a market the software industry has priced itself out of for thirty years. Where pain is this quantified and this persistent, the absence of a good solution is not an accident — it has structural causes, which Chapter 9 dissects.

The gaps are also countable. Each research lens was forced to defend its candidates against the strongest incumbent before accepting them, and what survived that screening is a measurable body of work: five to eight validated gap deep-dives per lens, thirty-two in total across the five tracks, distilled into the twelve ranked opportunities of Chapter 10. Figure 2 shows how the survivors distribute across the lenses — and why the plumbing and missing-links lenses, which ask what is broken underneath and between working systems, carry the deepest concentrations of unbuilt software.

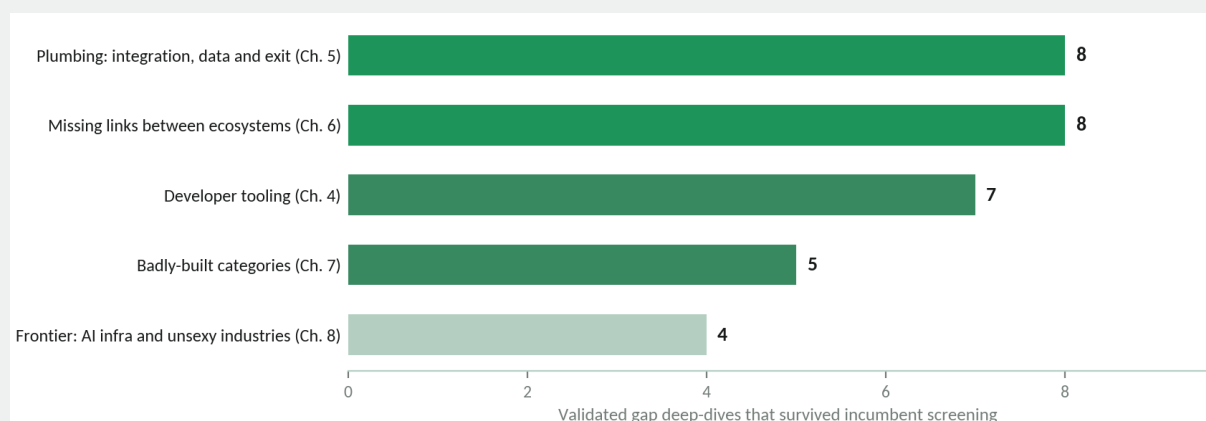


Figure 2. Validated gap deep-dives per research lens after incumbent screening (counted from Chapters 4-8)

4. Developer Tooling: Seven Validated Gaps

Developer tooling is the most crowded software market on earth, which is exactly why the surviving gaps here are interesting: they persisted through two decades of intense competition. Each of the seven gaps below was checked against its incumbent (Docker and Nix for environments, Playwright retries and BuildPulse for flakiness, Datadog and Grafana for observability, Confluence and Notion for documentation, GitHub review flows, Dependabot and Renovate for dependencies). Each incumbent solves an adjacent problem, and the unsolved remainder is the gap.

4.1 Flaky tests: detection exists, root-cause repair does not

Every serious tool can detect and retry a flaky test; almost nothing on the market can say why it flakes, let alone fix it. A practitioner writing in April 2026 reported roughly 4% of a 1,200-test E2E suite failing spuriously per run, with every failure triggering "a re-run, a triage, a Slack thread" (dev.to, Apr 2026). The industry default fix is the retry loop, described bluntly as a band-aid (Medium, May 2026), and reproducing a flake still requires disabling retries and hand-tuning (charpeni.com, Mar 2025). The gap is newly urgent because LLM-application evals are non-deterministic by construction — "part of your code now returns a different answer every time you run it" (kig.re, Jun 2026) — so every team adopting AI is newly exposed to flake-class CI pain (BuildPulse, Jul 2026). Causal attribution across CI telemetry, traces, and timing distributions is now technically feasible; the product that clusters flake signatures, replays the failing case under instrumentation, and files a diagnosis PR is unbuilt.

4.2 Observability is priced for enterprises, not for the 2-20 person team

Ask what monitoring a small team should use and the honest answer is either a free tier that ends abruptly or a Datadog bill that causes "bill shock" — the phrase is SigNoz marketing copy (Oct 2025), and an entire competitor genre exists to answer "is Datadog worth it?" (uptrace.dev, Jan 2025). More than thirty alternatives were catalogued in a single 2026 roundup with no default winner (hyperping, Jul 2026), which signals an unsolved category rather than a solved one. The missing product is not another backend: it is turnkey, flat-priced, sane-default observability for applications with fewer than ten services, where cost tracks team size instead of cardinality accidents. With OTel standardized, eBPF auto-instrumentation mature, and object storage making retention nearly free, the ingredients for a "you will never get a surprise bill" appliance finally exist.

4.3 Living documentation: nothing goes red

Documentation has no equivalent of a failing test. "Documentation goes stale silently. Unlike code, there is no test suite that goes red when your runbook references a deprecated service" (Atlassian Community, Feb 2026) — and the cost is measurable: new hires take two to three months longer to reach productivity when documentation is not current (GetDX 2025 study, via dosu.dev, Mar 2026). Freshness badges show age, not incorrectness; 2025-era "ask your docs" RAG bots make rot worse by confidently answering from stale pages. The gap is the missing checker: a tool that resolves every entity in a document — service names, endpoints, environment variables, runbook links — against the live repository and cluster, and posts a red mark on the pull request when a document references something deleted or renamed. LLM-based claim extraction just made the cross-referencing step cheap for the first time.

4.4 Code review does not scale to the AI-code era

Review was already the bottleneck before AI multiplied the volume: developers spend about six hours per week on review, roughly 15% of a working week (Microsoft Research, via actual.ai, Mar 2026), and the median engineer at a large company takes thirteen hours to merge a pull request (State of Code Review 2024, via graphite.com). Automated review bots demonstrably slow merges down — average PR closure duration increased from 5h52m to 8h20m after adoption (arXiv, Dec 2024) — while trust in AI output fell from 40% to 29% in a single year (Stack Overflow 2025 Developer Survey, via CIO, Aug 2026). The unsolved part is intent-level review: whether a change does what it claims, is safe to ship, and breaks invariants nobody wrote down. The buildable artifact is a machine-generated "verification dossier" attached to each PR — what the diff claims versus what the tests actually prove — so the human spends their fifteen percent on judgment rather than comprehension. Figure 3 collects the three measurements that define the bottleneck.

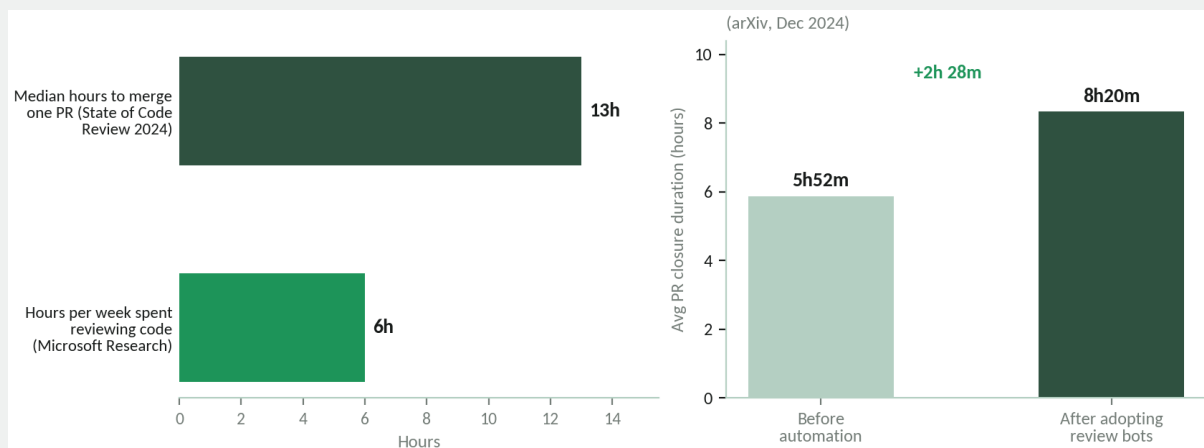


Figure 3. The review bottleneck: time invested, time to merge, and what automated review bots did to closure duration (sources as labeled)

4.5 Dependency maintenance: updates are automated, judgment is not

Renovate and Dependabot can open fifty update pull requests a week, but deciding which are safe remains manual, and the blast radius is now worm-scale. The Shai-Hulud npm worm compromised over 500 packages in September 2025 (StepSecurity, Sep 2025; CISA advisory, Sep 2025); a second wave poisoned more than 800 packages and over 25,000 GitHub repositories in December 2025 (backslash.security, Dec 2025); and an ACM study found that 2.30% of dependency updates carry behavioral, invisible breaking changes (dl.acm.org, Jul 2024). Nothing on the market evaluates an upgrade's behavioral risk — whether the API changed semantics despite a version bump, whether the new version phones home — before merge. Sandboxed execution is now a commodity, so a CI gate that installs both versions in isolation, diffs exported APIs, network calls, and install scripts, and posts a human-readable verdict is newly buildable.

4.6 Dev environments: containers exist, verified onboarding does not

Every team has a containerfile or a flake, yet "clone to running in ten minutes" still does not exist as a product. The unsolved slice is composition: toolchain, services, secrets, and seed data wired together and verified as one unit, with a self-test that fails with the exact fix command. The pain is diffuse but large — the same GetDX 2025 study that quantified documentation rot attributes two to three months of delayed productivity to onboarding friction, and the record adoption of LocalStack as a cloud-emulation workaround (AWS itself recommends it to cut dev costs, Oct 2025) is evidence that the underlying problem persists. No tool currently treats the developer environment as something with a test suite; an "env doctor" that CI runs on every pull request is the missing shape.

4.7 Test data: a third of engineering time spent fabricating data by hand

Teams spend 30-40% of their time on data preparation for testing (getautonoma.com), production data is restricted and masking is slow (platformengineering.org, Dec 2025), and the state of the art in 2026 is still "a seed script or a Faker-based fixture" (seedfa.st, Apr 2026). Synthetic-data vendors (Tonic, synthesized, mostly.ai) sell almost exclusively to regulated enterprises at six-figure prices, leaving small teams unserved. What is missing is the one-command version: point the tool at a Prisma or SQLAlchemy schema and receive a versioned, referentially-consistent ten-thousand-row dataset with generated edge cases, refreshed on every pull request. Schema-aware LLM generation makes the per-schema modeling cost — historically the reason this never productized — collapse to near zero.

Two candidate gaps from this track were screened out and are worth recording as solved: static analysis and formatting (settled by linters, formatters, and LSP), and feature flags (a crowded space with several dominant platforms). CI itself generates complaints but has clear winners; it is friction, not vacuum.

5. The Broken Plumbing: Integration, Data and Exit

The second lens looks underneath working systems. The consistent finding is that connectivity is no longer the problem — APIs, connectors, and iPaaS platforms are everywhere. What is missing is ownership: nobody is paid to notice when the connection between two systems breaks, drifts, or silently loses data. Track R2 synthesized eight validated gaps in this layer, and they share one failure mode: every layer of the modern stack presumes an operator — a data engineer, a platform team, an IT department — that companies below roughly one hundred employees simply do not have. The price of that presumption is not abstract: four independent cost studies put it between \$58 for a single expense report and \$670,000 added to a healthcare breach, as Figure 4 shows.

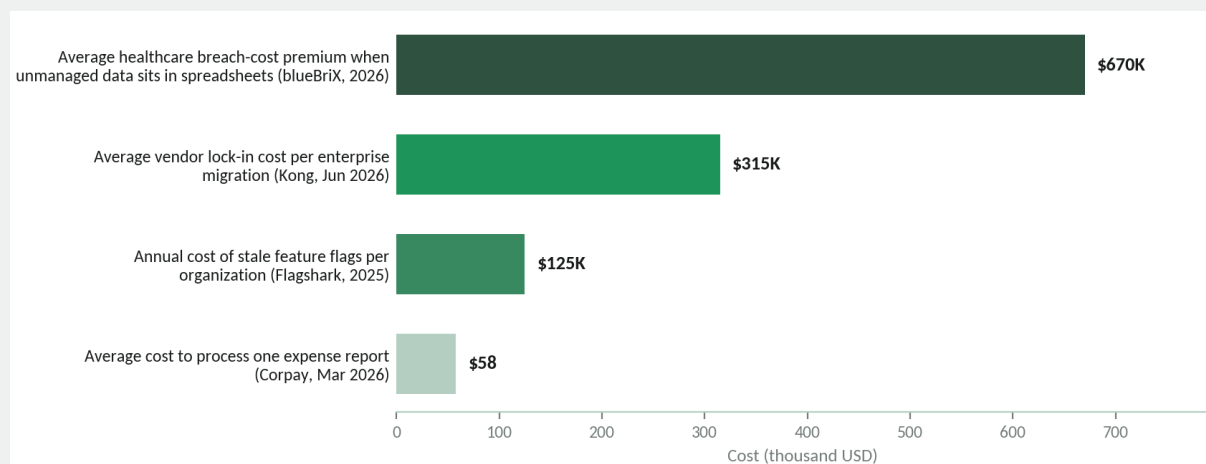


Figure 4. What unowned plumbing costs, in four independent measurements (thousand USD; sources as labeled)

5.1 Integration debt: unowned wiring that breaks silently

Companies run on point-to-point integrations built under deadline pressure, and the accumulated "integration debt" breaks silently at two in the morning (Bluepes, Apr 2025; salable.app, Feb 2026). Zapier and Make meter by the task, which punishes volume; Workato-class platforms start above \$50,000 and assume an expert operator; unified APIs collapse to the lowest common denominator. None of them watch the pipeline after setup — no drift alerts, no replay, no failure ownership. The pressure is compounding: task-specific AI agents are projected to integrate with 40% of enterprise applications by the end of 2026, up from under 5% in 2025 (Gabe Veatch, LinkedIn, Apr 2026), which multiplies the number of brittle pairwise links exactly when no one is watching them. Figure 5 sets that incoming wave of agent wiring against the CRM blindness it compounds: an eightfold rise in integrated applications, landing on a system of record that already loses 79% of its opportunity data.

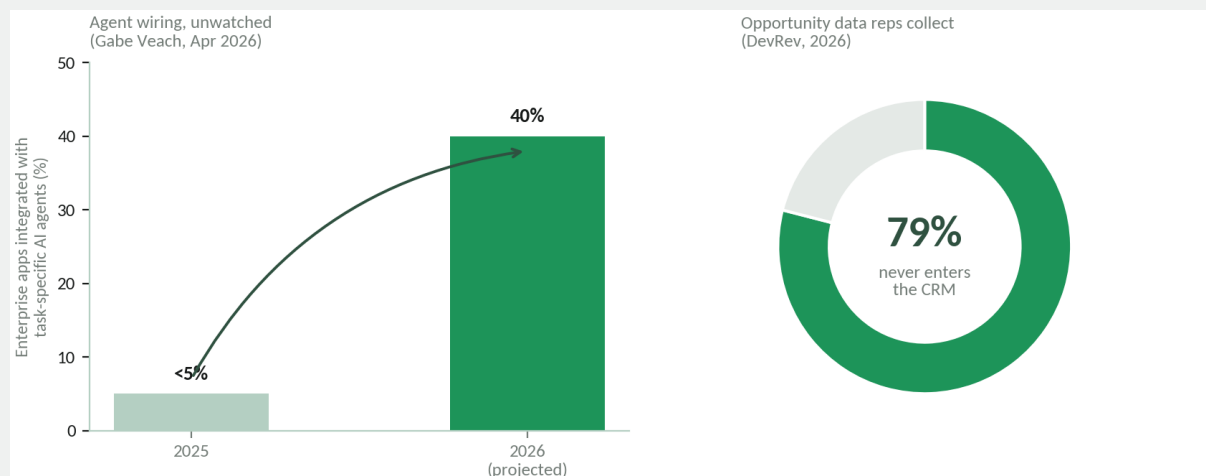


Figure 5. The unwatched wiring wave: projected agent integration share 2025-2026 (left) and the CRM data blindness it lands on (right)

5.2 The SMB data layer: the modern data stack ends at employee ~100

The Fivetran-Snowflake-dbt-BI stack presumes an analytics engineer; below that threshold, companies revert to CSV email attachments and "ductaped spreadsheets." Seventy data leaders and practitioners agree the current data stack is too complex (Medium, Mar 2025), and 63% of practitioners spend more than 20% of their time simply coping with data-stack complexity (The Modern Data Company, Aug 2025). The 2025-26 wave of "AI analysts" inherits exactly this fragmented connector landscape and its stale, permission-wrong data. The open slot is an opinionated, fully managed data layer for one vertical — normalize Shopify, ads, ERP, and shipping into one owned schema, exposed as both metrics and an API that the company's agents can query.

5.3 Exit and migration tooling: leaving is archaeology

\$315K

average vendor lock-in cost per enterprise migration; 62% of organizations cite data migration as the ERP failure driver (Kong, Jun 2026; companieshistory.com, Jun 2026)

Regulators are killing egress fees — the EU Data Act's portability obligations became applicable in September 2025 — but the mechanics of leaving remain hand-built: destination vendors build generous, shallow import wizards, and nothing exports workflow logic, the two hundred saved automations and permission model that constitute the real lock-in. The extreme case is ERP, where mid-size projects

average \$7.1 million and 17.4 months with a 68% failure rate (Panorama 2025, via [kreativecoretech.com](https://www.kreativecoretech.com)), and where 62% of organizations name data migration as the failure driver. Even trivial exits are scrambles: Notion Mail's 2026 sunset gave users weeks to export while connected syncs broke (TechCrunch, Jun 2026). Migration-as-a-service for one fleeing path — a dying platform to its rival, data plus workflow translation at a fixed fee — has no dedicated vendor, and regulation is now creating the demand.

5.4 Identity syncs accounts, not permissions

SCIM provisions login accounts, but each application's internal permission graph drifts silently — offboarding removes the login, not the access. The capability is widespread but gated: most SaaS applications support SCIM "but you can't [use it] without an enterprise plan" (stitchflow.com, Jan 2026), and where it is absent, IT teams hand-roll workarounds (ssotax.org). The enterprise answer, SailPoint-class identity governance, costs six figures and assumes an IT department. Nothing reads the role-and-permission graph inside each application — many have no API for it at all. Browser automation plus LLMs newly make a read-only "permission census" scanner buildable: crawl each admin console, diff permission state over time, alert on orphaned access, and emit the audit table that SOC 2 reviewers keep asking for.

5.5 Legacy modernization: the crisis is knowledge, not code

Between 220 and 800 billion lines of COBOL still settle roughly 80% of the world's financial transactions, maintained by a generation now walking toward retirement (phasechange.ai, Oct 2025). AI code translation attracts the headlines — a single blog post about AI COBOL translation briefly wiped some \$30 billion off IBM's market capitalization (devops.com, Feb 2026) — but "translating code isn't the same as modernizing," because the missing artifact is the recovered business rule, not the syntax. The long tail is mid-market: AS/400, Delphi, and Access line-of-business applications with no tooling and no staff to validate a rewrite. The sellable object is the executable behavior spec — a characterization test suite extracted by interviewing domain experts — sold independently of any rewrite.

5.6 The spreadsheet is the world's actual data layer

Eighty-five percent of construction professionals still use Excel for estimating and costing (premiercs.com, 2026); unmanaged patient data in spreadsheets adds an average of \$670,000 to healthcare breach costs ([bluebriX](https://bluebriX.com), 2026); and inter-company exchange still runs on nightly SFTP drops and EDI batches with per-client custom scripts (filefeed.io, Feb 2026). Two decades of "Excel killers" failed because they demanded greenfield rebuilds that lose fifteen years of formulas and macros — so users keep shadow copies and the "system" bifurcates. The missing product absorbs the workbook instead of replacing it: point it at the operational spreadsheet, auto-generate the schema, change-tracking, audit log, and API around the same file, and let humans keep Excel as the interface. LLMs can finally read a gnarly workbook — formulas, structure, VBA — and infer the schema underneath.

5.7 Real-time plumbing is enterprise-only by construction

Kafka-class streaming is architecturally correct but operationally absurd for small teams — "overkill for simple task queues" (inteca.com, May 2025) — and teams flee it over broker cost, storage growth, and rebalancing delays (automq.com, Jun 2026). A whole 2025-26 product category exists just to simplify it. Small firms do not need a platform; they need the outcome: a record changed, downstream applications see it now, idempotently, with retries. Durable-execution primitives and queue-as-a-service finally make "webhooks with guarantees" — exact-once fan-out, replay, a dead-letter dashboard, schema versioning — the shippable 10% of Kafka that 90% of small integrators actually need.

5.8 Inter-company integration still runs on batch files

B2B SaaS vendors hand-roll per-client file ingestion with no tooling: monitored SFTP folders, bespoke CSV and XLSX parsers, and reconciliation over email. Enterprise EDI gateways are priced for the Fortune 500, and iPaaS connectors target APIs rather than the arbitrary files clients actually send. The per-client custom-script tax hides inside professional-services line items, which is precisely why no standalone product ever formed. Arbitrary-file schema inference is newly cheap, so "drop-box to database" — monitored inbox in, auto-detected schema, validated rows out via API — sold per client per month to the SaaS vendor rather than the end user, is now a product where before it was a services engagement.

The synthesis of this chapter is one sentence: **connectivity exists; ownership does not**. Entry is funded everywhere (generous import wizards, free tiers, onboarding teams) while exit and reliability are funded nowhere — and the agent wave is about to inherit exactly this unmaintained wiring.

6. Missing Links: Connective Tissue Never Built

A missing link is defined precisely: two things that both exist and both work, but are not connected. Building the connective tissue — rather than improving either endpoint — unlocks outsized value, because the endpoints already have entire industries maintaining them. Track R3 evaluated twelve candidate links and discarded four as already being built (API contract testing, design-token format interop, compliance-as-code, and local-file-to-cloud bridging). The eight survivors below share a single historical cause: the join between the two representations always cost more than the failures it prevented. That equation has now flipped.

6.1 Design intent and shipped production code

Figma files are the source of design truth; running production interfaces are the source of behavioral truth; and the two drift apart the day after every handoff. In Figma's own 2025 State of the Designer report, 91% of developers and 92% of designers agree the design-to-code handoff still needs improvement. Every existing tool is either a one-shot converter (which bridges once, then drift resumes) or screenshot-to-screenshot visual regression (which detects change, not drift from design intent). The newly buildable connector is continuous "design drift CI": render the deployed interface, diff it against the linked Figma frames at token and component granularity, and post a reviewable drift report on every pull request. The W3C Design Tokens Format Module reached a stable specification in October 2025, giving the diff a machine-readable common layer for the first time, and vision-capable models can now attribute differences to specific tokens rather than emitting a wall of pixel noise.

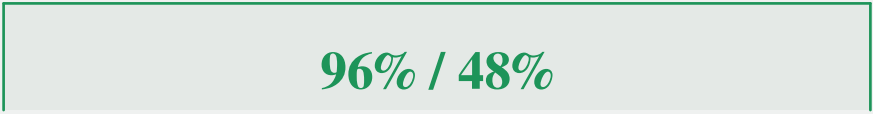
6.2 Specs and tests: executable requirement traceability

Requirements traceability survived only inside regulated-industry ALM suites priced for aerospace and medical budgets; mainstream agile teams dropped it entirely rather than pay the tax. Now spec-driven development is surging precisely because AI agents need machine-readable specifications to write code against (martinfowler.com, Oct 2025) — but the trace runs one way, spec to code. Nothing checks the return path: that every requirement has an assertion, and that every test is justified by a requirement. When agents write most of the code, the specification becomes the only human-reviewed artifact, and a "trace linter" — coverage.py for requirements instead of lines, failing CI on orphan requirements or orphan tests — becomes the audit trail for the whole pipeline. The wedge into the market is the fast-growing spec-driven-development community itself.

6.3 Docs and runtime: self-verifying documentation

This is the same wound as gap 4.3, seen from the link side: documentation and deployed behavior are two representations of one system with no verifier between them. Guides to documentation drift recommend discipline, not verification (mintlify.com, Jun 2026); freshness badges measure age, not incorrectness; AI doc generators snapshot behavior and then drift one generation later. The mechanical version — extract every checkable claim from a document, verify it against code, OpenAPI schemas, or a live staging environment, and file a precise "this section is now false" report naming the offending commit — only became a mechanical task with current language models. Start with READMEs, where the pain is universal and the land is open source; monetize enterprise runbooks, where being wrong is most expensive.

6.4 AI-generated code and the verification trust layer



96% / 48%

of developers do not fully trust AI-generated code (96%), yet only 48% always verify it before committing — the behavioral gap a merge-gate ledger closes (SonarSource, Jan 2026)

Code review assumes the author understood what they wrote; machine-generated code breaks that assumption, and the safety net — linters, tests, static analysis — certifies syntax and coverage, never provenance or comprehension. The numbers describe a systemic risk rather than a preference: 96% of developers do not fully trust AI-generated code while only 48% always verify it (SonarSource, Jan 2026); usage rose to 84% while trust fell eleven points in a year to 29% (Stack Overflow 2025). No attestation standard exists for code authorship — C2PA covers media, SLSA covers build provenance, and neither answers the question a tech lead actually asks: which lines were machine-written, were they ever run, and does the human sign-off understand them. A verification ledger at the merge gate — machine-readable attestations bound to every agent-authored pull request, with a per-repository dashboard of trusted-versus-verified — is the CI/CD of the agent era, and the field is empty. Figure 6 sets the two survey pairs side by side: adoption rising while trust collapses, and distrust near-universal while verification stays minority behavior — the exact spread a merge-gate ledger converts into a product.

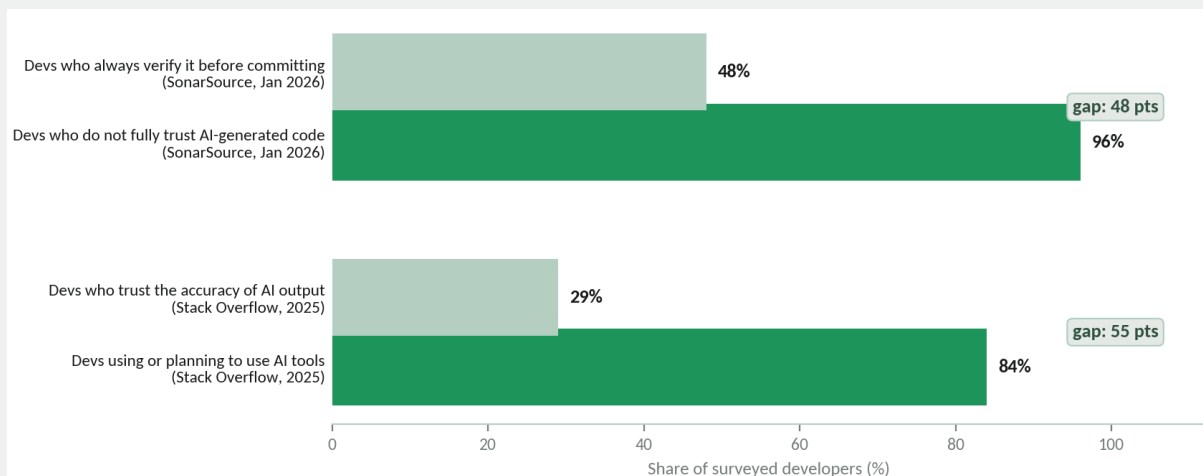


Figure 6. The verification gap: AI tooling adoption versus trust in output, and distrust versus actual verification behavior (sources as labeled)

6.5 Deploys and incidents: the missing final rung

Ticket-to-commit-to-deploy linking exists inside individual vendors (Azure DevOps documents work-item linking; Jira plus GitHub Actions visualize deploys). The rung that would answer "which change caused the outage" — the deploy-to-incident causal join — is manual everywhere: root-cause timelines are reconstructed by hand, under stress, after the fact, which is why postmortems read like autopsies (hyperping.com, Apr 2026). The four data types live in four vendors, each pairwise integration is shallow, and correlating deploys to regressions requires statistical joinery nobody wanted to own.

CDEvents and OpenTelemetry now normalize deploy events, and warehouse-native analytics makes the cross-vendor join trivial — a "change ledger" that auto-correlates and auto-drafts the suspected-change section of every postmortem sells directly to the incident commander.

6.6 Feature flags, analytics, and code: the retirement ledger

Stale flags are quantified technical debt — more than \$125,000 per organization per year (flagshark.com, Jun 2025) — and every flag vendor now publishes a flag-debt guide, which means the debt is endemic and self-inflicted. Flag platforms and analytics platforms remain two systems joined manually per experiment: nothing ties a flag's decision to the business number that justifies keeping or killing it, because flag evaluation lives in application runtime while outcome data lives in the warehouse, and nothing stamps events with the flag state that produced them. The buildable shim stamps all telemetry events with active flag states, then builds the automatic ledger — flag to code paths to outcomes to a retirement recommendation ("kill flag X: saves three thousand lines, no metric moved in ninety days").

6.7 CRM context and engineering systems: tenant observability

The support-escalation ritual is pure waste: a ticket opens, a support engineer exchanges messages trying to reproduce an issue (leaddev.com, Feb 2026), while the specific facts — which version this tenant runs, which flags are active for them, which recent deploy touched their workflow, which errors their requests produced — sit in engineering systems one join key away. The join key (tenant identifier) exists on both sides and was never propagated through telemetry. Multi-tenant SaaS is now the default and OpenTelemetry supports tenant attributes, so the missing link is a middleware that captures per-account runtime snapshots and injects them into Zendesk or Intercom tickets automatically — observability aimed at the support queue instead of the dashboard.

6.8 Formal methods and mainstream development

Formal verification is mature, proven, and fringe — kept out of the mainstream by the one cost that mattered: writing formal properties cost more engineer-time than the bugs it prevented. Martin Kleppmann's December 2025 argument is that AI collapses exactly that translation cost. The mainstreamable subset does not require theorem proving: a "property copilot" that reads a pull request, proposes a handful of prose-derived invariants ("money is never created or destroyed in a transfer") as runtime monitors, executes them in staging and production, and flags violations with the trace. Runtime verification as a service is the property gap between what tests check and what types guarantee — and it is now fillable.

Discarded as already building, for the record: API contract testing (a mature ecosystem and a reported \$2.8B services market in 2025); design-token format interop (the W3C stable format plus Style Dictionary closed it); compliance-as-code (Drata, RegScale, and Ketrix already sell it); and local-files-to-cloud bridging (real pain, but fragmented with no small-team wedge).

7. Badly-Built Categories: Why Hated Software Stays Bad

Some categories have hosted hated software for ten to twenty years while dozens of well-funded challengers failed to fix them. Track R4's core finding is that this is not an accident of execution but a set of compounding structural mechanisms — and that AI changes the economics of exactly one of them. Understanding which mechanism locks a category tells you whether a genuine fix is even possible, and what shape its distribution must take.

7.1 The four mechanisms

- **Buyer does not equal user.** The person selecting the software (IT, procurement, a VP) is not the person who suffers daily: "a bad UI doesn't impact sales, because the people who do the purchase don't use the application" (Reddit r/sysadmin, Dec 2025). Enterprise incentives are "entirely misaligned" (Medium, Jul 2026) — an observation HN was making in 2010 that remains true in 2025.
- **The software faithfully encodes a resented process.** "My biggest problem is not Jira itself, but how people are using it" (Ministry of Testing, Feb 2025); project-management tools "fail because of resistance, not because of missing features" (Breeze.pm, Oct 2025). Automating a hated process produces hated software, whatever the UX.
- **"Good enough" plus integration moats.** Legacy tools sit on SSO, workflows, exports, and muscle memory; enterprise systems historically carried data error rates above 50% and survived anyway, because replacing them is riskier than hating them (MIT Sloan Review, 2007, still cited).
- **Checklist procurement has no UX feedback loop.** Buying is checklist-driven, so vendors invest in the demo-to-checkout path, not the daily-use path; poor user adoption causes roughly 70% of digital-transformation initiatives to fail (UX Matters, Feb 2025).

The 2025-26 AI twist: for the first time the marginal cost of manual entry, summarization, and retrieval is collapsing — service-desk-class agents report up to 60% reductions in manual workload (BCG, Oct 2025). That attacks the data-quality half of the moats and the manual-entry layer of the hatred, but it does **not** fix the buyer-user mismatch, which is organizational rather than technical. The table summarizes which mechanisms AI unlocks.

Mechanism	Representative locked categories	Does AI (2025-26) unlock it?
Buyer is not the user	CRM, project management, intranets, expense	No — requires bottom-up distribution
Encodes a hated process	Project management, approvals, timesheets	Partially — agents can bypass process steps
Manual entry and retrieval cost	CRM, notes, timesheets, docs, search	Yes — this is the big unlock
Integration and permission moats	Search, email, intranets, PM tools	Partially — the connector layer is the new moat
"Good enough" trap and free defaults	Email, spreadsheets	Yes — via outcome pricing instead of seat pricing

Table 2. Which lock-in mechanisms does the AI era actually unlock?

7.2 CRM: the cleanest case in the industry

Salespeople spend roughly 60% of the week on non-selling activity (Attention, citing Salesforce's own numbers, Feb 2025), and 79% of the opportunity data reps collect never makes it into the CRM (DevRev, 2026). The rep's labor is the input; the manager's dashboard is the output; the rep receives zero personal return for typing — so they rationally stop typing, and CRM adoption failure rates of 20-70% follow (heydan.ai, Dec 2025). A decade of mobile apps, gamification, and mandatory-field policies treated the symptom. The genuine fix newly buildable: CRM as an observation system rather than an entry system — calls become transcripts become structured fields, with the rep reviewing rather than typing, sold rep-first so adoption is voluntary.

7.3 The expense-report dead zone

"We hate them. The companies that build expense management software know that we hate them" (New York Times, Nov 2024). Half of workers do not file some reports at all, abandoning an average of \$26.25 each (WalkMe, Dec 2023); a single report still costs \$58 to process and 19% contain errors (Corpay, Mar 2026). Corporate cards and OCR attacked the receipt; the residual pain is policy, out-of-pocket float, and multi-step approvals — org-process pain that software mirrors faithfully. The credible fix kills the report rather than improving it: card-first everywhere plus agents that assemble evidence and auto-approve in-policy spend, with finance buying exception-handling instead of review-of-everything.

7.4 Meetings: transcription is solved, execution is not

Unproductive meetings cost businesses on the order of \$375B annually (Fellow.ai, May 2025), and individual contributors now lose 3.7 hours per week to them, up 118% since 2019 (Asana data, via SpeakWise, 2026). AI notetakers finally crossed the adoption line — roughly 75% of professionals use one (Saner.ai, Jul 2026) — yet "the action items they extract stay yours to carry out, and each meeting

becomes its own silo" (Tana, Jul 2026). The unbuilt layer is the write-back: notes that flow into tickets, CRM, and documents automatically, with consent tooling for the new legal exposure the transcripts create (Mayer Brown, Jun 2026). Better transcripts are a commodity; the meeting-to-action loop is the product.

7.5 Email: a graveyard that teaches the real lesson

The email-client category is a museum of failed "fixes": Notion Mail shut down in September 2026 roughly a year after launch, discontinued in favor of AI agents (TechCrunch, Jun 2026), with more than half of its users never actually opening the inbox (Enterprisedna, Jun 2026); Superhuman, the celebrated UX play, ended up absorbed into Grammarly. The 2026 lesson generalizes to every category in this chapter: people do not want a faster version of the hated thing — they want less of the thing. Notion's own pivot is the tell: agents that drain the inbox (draft, schedule, extract tasks, unsubscribe) rather than clients that render it more beautifully.

7.6 Intranets, docs, and timesheets: the same shape

Fifty-seven percent of employees see no purpose in their company intranet (Firstup, Jun 2024) and up to 90% fail to deliver clear value (Coveo, via Nexinite, Aug 2025) — two masters, billboard and tool, serving neither. Documentation rot was quantified in Chapter 4; the wedge is generation from the work itself plus staleness detection. Time tracking punishes its user while benefiting the firm's billing, and costs law firms \$25,000-\$100,000 per year in unbilled time (LeanLaw) — passive reconstruction of the day from calendar, email, and documents, presented as a draft timesheet to edit, is the fix now that reconstruction quality crossed the usability threshold. Every one of these wedges has the same shape: sell the suffering user the tool that removes their work; sell the resulting clean data to the buyer.

8. Frontier Gaps: AI Infrastructure and Unsexy Industries

The frontier lens covers two very different territories: the plumbing that the AI application era is missing underneath itself, and the industries where the software industry never bothered to show up. The screening discipline matters most here, because the AI-infrastructure space moves fastest: LLM tracing was discarded as solved (nine or more mature competitors), prompt versioning was discarded (instant rollback is now standard), and local inference operations was deprioritized as commoditized by the vLLM and Ollama ecosystems. What survived is narrower and sharper.

8.1 Evals-in-CI: testing non-deterministic systems where engineers already live

Every AI feature currently ships with effectively no meaningful test coverage, because pytest asserts equality and model outputs are non-deterministic — "part of your code now returns a different answer every time you run it" (kig.re, Jun 2026). Existing eval platforms run offline datasets in their own dashboards, a manually-triggered world outside the pipeline engineers already trust, while flaky evals "wreck your CI signal" (BuildPulse, Jul 2026). The stakes are escalating fast, as the chart below shows: documented AI incidents rose from 233 in 2024 to 362 in 2025, and enterprise LLM API spend more than doubled from \$3.5B to \$8.4B in roughly six months. The missing product is a GitHub-Action-shaped statistical gate — fixed seeds, pass-at-k thresholds, regression bands against stored baselines — that makes "npm test" meaningful again when the model is stochastic. Distribution is native developer tooling: no sales force required.

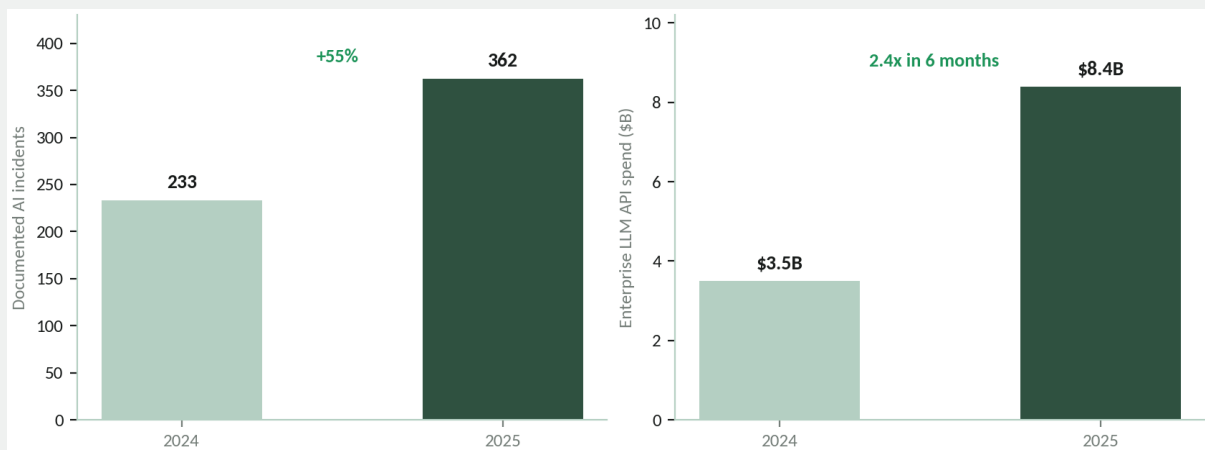


Figure 7. The AI era is escalating faster than its tooling: documented AI incidents (left) and enterprise LLM API spend (right)

8.2 Portable agent memory

Agent memory is becoming the most valuable user data since email, and it is trapped per vendor: "Mem0's memory is stored in their cloud infrastructure, creating a new form of vendor lock-in" (arXiv, May 2026). Open standards cover tool calls (MCP) and agent-to-agent messaging (A2A) but not persistent user memory; every chat history is already fragmented across providers with no interop. An open-source, local-first memory server with a documented export format and a thin hosted sync tier is a standards play — credibility-first, slow to monetize, and strategically positioned for the moment regulation turns memory portability into the next browser-data fight.

8.3 AI FinOps: cost allocation, not token counters

Gateways count tokens and observability platforms graph spend, but engineering still cannot answer "what did this feature cost us per customer this month?" — and agent loops make cost non-linear, since one bad prompt can burn a month of budget. With CFOs now seeing an AI line item and asking for

chargeback, the missing layer is FinOps-style allocation: tagging spend to feature, team, and customer; amortizing compounding agent-loop costs; and a policy engine that auto-downgrades or halts runaway agents. An open-source allocation and guardrail engine sitting on data teams already emit (gateways, OpenTelemetry) lands via the OSS route and expands into hosted policy enforcement.

8.4 Unsexy industries: construction sub-tier and freight brokerage

Twenty-seven percent of AECO professionals still rely on Excel and PDFs (PR Newswire, Jul 2025), and 85% of construction professionals use Excel for estimating and costing (premiercs.com, 2026) — while the existing tools target large general contractors at enterprise pricing, leaving the five-to-fifty-person subcontractors who actually bid the work unserved. Camera takeoff and email-to-estimate workflows newly remove the data-entry barrier that made sub-tier software uneconomical; the channel is trade associations rather than a field sales force. Freight brokerage is the parallel case: brokerages run on thirty-year-old TMS plus phone-email-Excel glue, and the actual money — per-lane margin visibility, carrier-risk scoring fed by live data — is tribal knowledge an overlay can now read from broker inboxes and TMS exports directly.

Gap	Sales force needed?	Why it wins for a small team
Evals-in-CI adapter	No — developer PLG	Rides existing CI rails; deterministic-statistics wedge; OSS to hosted
AI FinOps allocation	No — OSS + PLG	Sits on data teams already emit; policy engine is the moat
Code provenance attestation	No — compliance buyers, inbound	Standards-shaped greenfield; regulation tailwind
Portable agent memory	No, but slow monetization	Standards play; credibility first; pair with consulting
Construction sub-tier estimating	Eventually — association channel first	LLM takeoff removes the data-entry barrier; one trade, one region
Freight margin intelligence	Phone-sales motion, no field force	Overlay-not-replacement avoids the TMS sales cycle

Table 3. Shippability of the frontier gaps for a 2-4 person team

Honesty about coverage: field services, small-manufacturer MES, government legacy systems, elder care, and SMB supply-chain visibility were queued in the research plan but could not be source-verified within this pass due to search-quota rate limits. Prior expectation is that they are structurally similar to the two validated verticals — spreadsheet shadow systems below an enterprise pricing floor — but they are flagged as follow-up research rather than asserted as findings.

9. Cross-Cutting Patterns: Why These Gaps Persist

When forty-odd validated gaps from five independent tracks are laid side by side, six structural patterns explain nearly all of them. These patterns are the actual research finding — the individual gaps are instances. They also explain why the gaps are opening now rather than ten years ago, and why a small team can win several of them against incumbents who cannot respond.

9.1 The ownership vacuum

The single most repeated failure mode across all tracks: a layer works, and nobody is paid to watch it. Integrations break silently because the iPaaS monetizes task volume, not reliability; documentation rots because no test suite goes red; permission graphs drift because identity syncs accounts rather than access; the deploy-to-incident join is manual because the four data types live in four vendors, each with shallow pairwise integrations. Every one of these is a monitoring or verification layer sold to the person who gets paged — a buyer who feels the pain weekly and can approve a small purchase without a committee.

9.2 Every layer presumes an operator

dbt presumes an analytics engineer; Kafka presumes a platform team; SailPoint presumes an IT department; mainframe modernization presumes a consultancy. Below roughly one hundred employees those people do not exist, so the layer silently reverts to spreadsheets, email attachments, and nightly SFTP drops — the "actual data layer" of the world economy. The products that win the SMB tier will be the ones that package the operator's judgment as defaults, not the ones that offer the operator's tools at a lower price.

9.3 Entry is funded, exit is not

Every vendor builds generous import wizards; none productizes the exit. The result is a \$315,000 average cost per migration, a 62% data-migration failure driver for ERP projects, and a regulations-versus-tooling gap now opening under the EU Data Act. Migration-as-a-service for specific fleeing paths is structurally under-supplied because exit buyers are one-time and awkward — which is exactly why the niche remains empty and why referral deals with destination vendors can substitute for a recurring sales motion.

9.4 The translation-cost collapse

The meta-pattern of Chapter 6: nearly every durable missing link — design to code, spec to test, prose to formal property, ticket to context, workbook to schema — stayed unbuilt because translating between two representations cost more engineer-time than the failures it prevented. Language models collapse precisely that cost. This is the "why now" for half the gaps in this report, and it is also a screening

heuristic for any additional gap candidate: if the blocker was translation between representations, it is now buildable; if the blocker was a buyer-user mismatch or a distribution problem, AI does not help.

9.5 Buyer-user mismatch survives the AI era

Chapter 7's deepest lesson deserves repetition at the pattern level, because it is the one AI does not fix. Categories where the suffering user has no purchasing power and the vendor's revenue depends on the pain's byproduct — data, control, compliance — regenerate their badness through every technology cycle. The 2026 distribution answer is bottom-up: give the rep, the developer, the support agent a tool that removes their unpaid work, and sell the clean exhaust data upward. Notion Mail's shutdown is the proof that "a better version of the hated thing" loses to "the hated thing, deleted."

9.6 Overlay beats replacement for small teams

The gaps a two-to-four person team can actually win are overlays on incumbents rather than replacements of them: the evals adapter rides GitHub Actions rather than competing with eval platforms; the AI-FinOps engine sits on gateway and OpenTelemetry data rather than replacing the gateway; the freight margin overlay reads the TMS rather than replacing it; the integration observability proxy watches the wiring rather than rewiring it. Replacement sales cycles need field forces and years; overlay products sell to the person in pain, in a self-serve motion, while the incumbent cannot retaliate without cannibalizing its own platform revenue.

10. The Build Shortlist: What a Small Team Could Ship

This chapter converts the research into a ranked shortlist. The ranking weighs four factors: strength of the evidence that the gap is real; openness of the field (no dominant solution within reach); whether the enabling shift has actually happened rather than being promised; and realism of the distribution model for a team of two to four people with no field sales force. Twelve opportunities cleared the bar. The top three deserve detail before the table.

10.1 First: the evals-in-CI adapter

It addresses the sharpest unmet need found in the entire research — every AI feature shipping with no meaningful test coverage, inside a market whose incident count and spend are both doubling (Figure 7) — and it has a native developer distribution path. The product is narrow on purpose: a CI component that wraps existing eval suites in deterministic statistics (fixed seeds, pass-at-k thresholds, regression bands against stored baselines) and posts an ordinary green-or-red check on the pull request. It does not compete with eval platforms; it connects them to the workflow engineers already trust. That narrowness is

the moat: the big platforms monetize dashboards and judge-token spend, so none of them is structurally motivated to make the CI adapter excellent.

10.2 Second: the verification ledger for AI-generated code

The 96-versus-48 trust gap is a quantified, growing tax on every engineering organization, and no attestation standard exists for code authorship — the field is empty at exactly the moment regulation (the EU AI Act phase-in and secure-software attestation momentum) begins demanding provenance records for machine-generated artifacts. The buildable first version is a Git attestation layer: per-commit metadata recording AI-assistance level, binding to the tests that actually ran, and an "understood-by" sign-off, surfaced as a pull-request badge and an exportable audit report. Landing via an open-source CLI makes adoption voluntary and viral inside engineering orgs; the compliance buyer arrives later and pays enterprise prices.

10.3 Third: the integration observability proxy

Chapter 5's ownership vacuum in product form: a read-only proxy that sits in front of the customer's existing Zapier wiring, webhooks, and exports, alerts on schema drift, empty payloads, and dropped events, and offers one-click replay with diff reports. It requires no migration, sells to whoever gets paged when the sync dies, and rides the agent-integration wave (under 5% to a projected 40% of enterprise apps within 2026) that is about to multiply the amount of unwatched wiring in the world. The wedge is deliberately unglamorous — which is why no incumbent competes for it.

#	Opportunity	Lens	Why the field is open	First ship
1	Evals-in-CI adapter	AI infra	Eval platforms monetize dashboards, not the CI gate; no agreed green/red semantics exist	GitHub Action wrapping existing eval suites
2	AI-code verification ledger	Links	No attestation standard for code authorship; regulation creating demand	OSS CLI + PR badge
3	Integration observability proxy	Plumbing	iPaaS monetizes volume, not reliability; nobody watches post-setup	Read-only proxy with replay
4	Self-verifying docs (doc tests)	Tooling/Links	Freshness tools measure age, not truth; RAG made rot worse	README checker, OSS-first
5	Flaky-test root-cause repair	Tooling	Everyone detects and retries; nobody diagnoses or fixes	CI plugin filing diagnosis PRs
6	CRM as observation system	Categories	79% of data never entered; entry-system model is structurally dead	Call capture to structured fields, rep-first
7	AI FinOps allocation and guardrails	AI infra	Token counters everywhere; allocation and runaway-agent policy nowhere	OSS engine on gateway/OTel data
8	Permission census scanner	Plumbing	SCIM is enterprise-gated; permission graphs invisible	Headless-browser audit crawler

#	Opportunity	Lens	Why the field is open	First ship
9	Spec-to-test trace linter	Links	SDD surge is one-directional; return path unchecked	GitHub Action for SDD repos
10	Flat-priced small-team observability	Tooling	Enterprise pricing punishes small teams; 30+ alternatives, no winner	One binary, one price, OTel-native
11	Construction sub-tier estimating	Unsexy	Tools priced for GCs; 85% of estimating still in Excel	One trade, camera takeoff, association channel
12	Excel absorption layer	Plumbing	Killers demand rebuilds; nobody absorbs the workbook	Schema + audit log around the live file

Table 4. Twelve ranked opportunities from the research

Two disciplines should govern any entry into this list. First, verify the gap again at build time with fresh searches — the AI-infrastructure rows move fastest, and a category that was open in early 2026 may be closing by late 2026. Second, choose the overlay shape wherever one exists: the pattern across all five tracks is that small teams win by watching, verifying, or connecting systems that incumbents built and then neglected — not by asking anyone to rip anything out. The gaps are real, quantified, and structural; the scarce resource is the willingness to build unglamorous plumbing in markets where nobody else is paid to care.

Appendix A: Source Register

Key sources by research track, as cited in the text. All statistics in this report were seen directly in these sources during the research window; none were invented or extrapolated. Older statistics are flagged in the text where they appear.

Developer tooling (Track R1)

dev.to (Apr 24, 2026). Playwright and the fix for flaky tests. — dev.to (May 2026). Flaky tests as the slow leak in automation pipelines.

currents.dev (Oct 2025). Flake patterns over time. — buildpulse.io (Jul 31, 2026). Flaky LLM evals in CI.

SigNoz (Oct 2025); uptrace.dev (Jan 2025); hyperping (Jul 2026); motadata (Jun 2026). Datadog pricing and alternatives genre.

Atlassian Community (Feb 2026). Confluence wikis confidently giving wrong answers. — dosu.dev (Mar 6, 2026), citing GetDX 2025: 2-3 months delayed productivity from stale docs.

actual.ai (Mar 2026), citing Microsoft Research: ~6 hrs/week in review. — State of Code Review 2024 via graphite.com: median 13 hrs to merge.

arXiv (Dec 2024). Automated Code Review In Practice: closure duration 5h52m to 8h20m. — Stack Overflow 2025 Developer Survey via CIO (Aug 2026): trust 40% to 29%.

StepSecurity (Sep 15, 2025) and CISA (Sep 23, 2025) on Shai-Hulud; backslash.security (Dec 3, 2025) on the second wave; dl.acm.org (Jul 2024): 2.30% behavioral breaking changes.

getautonoma.com: 30-40% of time on data prep. — platformengineering.org (Dec 4, 2025). — seedfa.st (Apr 10, 2026).

Integration, data and exit (Track R2)

Bluepes (Apr 2, 2025). The hidden costs of poor integration. — ezintegrations.ai (Jun 11, 2026). Enterprise integration crisis.

Gabe Veach, LinkedIn (Apr 20, 2026). Agent integrations: under 5% (2025) to 40% (2026) of enterprise apps.

The Modern Data Company (Aug 21, 2025): 63% of practitioners spend over 20% of time on stack complexity. — Medium (Mar 13, 2025): 70 data leaders on stack complexity.

Kong (Jun 30, 2026): \$315K average lock-in cost per migration. — companieshistory.com (Jun 12, 2026): \$7.1M / 17.4-month mid-size ERP; 62% data-migration failure driver. — Panorama 2025 via kreativecoretech.com (Jul 11, 2026): 68% failure.

yeshid.com (Sep 11, 2025) and stitchflow.com (Jan 28, 2026) on SCIM gating; ssotax.org on workarounds; accessowl.com (2026) on the SSO/SCIM tax.

phasechange.ai (Oct 15, 2025): 220-800B lines of COBOL, 80% of financial transactions. — devops.com (Feb 25, 2026): IBM \$30B market-cap move on AI COBOL translation. — metaintro.com (Mar 17, 2026): \$125,525 average salary, +15% demand.

premiercs.com (2026): 85% of construction estimating in Excel. — blueBriX (2026): \$670K average breach-cost premium for spreadsheet PHI. — filefeed.io (Feb 28, 2026) on per-client file ingestion.

Missing links (Track R3)

Figma 2025 State of the Designer (cited via figmatoazure.com, Mar 2026): 91% of developers and 92% of designers say handoff needs improvement. — overlayqa.com (May 2026) and figr.design (May 2026) on drift.

W3C Design Tokens Community Group: Design Tokens Format Module v2025.10, stable Oct 28, 2025 (via zeroheight.com, Nov 2025; digitalapplied.com, Jun 2026).

SonarSource (Jan 2026). The AI trust gap: 96% do not fully trust, 48% always verify; via theregister.com (Jan 9, 2026) and thenewstack.io (Feb 2026). — stackoverflow.blog (Feb 2026): 84% usage, 29% trust, 46% active distrust.

hyperping.com (Apr 2026). Postmortems as autopsies. — learn.microsoft.com (Mar 2026) DevOps traceability docs; kinto-technologies.com (Oct 2024) Jira+GitHub Actions.

flagshark.com (Jun 2025): \$125K+ per year in stale-flag debt. — configcat.com (Aug 2026) on flags-versus-analytics split brain.

martinfowler.com (Oct 2025) on spec-driven development. — augmentcode.com (Apr 2026). — arXiv 2602.00180 (Jan 2026) on specs as executable contracts.

leaddev.com (Feb 2026). The escalation ritual. — martin.kleppmann.com (Dec 2025): AI will bring formal verification into the mainstream.

dataintelo.com (2025): contract-testing-as-a-service market at \$2.8B — basis for discarding that candidate.

Badly-built categories (Track R4)

Attention (Feb 2025), citing Salesforce: 60% of seller time on non-selling. — DevRev (2026), citing Salesso: 79% of opportunity data never entered. — BusinessWire (Aug 2026). The modern salesperson as data-entry clerk.

New York Times (Nov 3, 2024). Nobody likes doing expense reports. — WalkMe (Dec 2023): half of workers do not file; \$26.25 average abandoned. — Corpay (Mar 2026): \$58 per report, 19% error rate.

Fellow.ai (May 2025): \$375B annual cost of unproductive meetings. — Flowtrace (2025): \$29,000 per employee per year. — Asana via SpeakWise (2026): 3.7 hrs/week, +118% since 2019.

Saner.ai (Jul 2026): ~75% AI notetaker adoption. — Tana (Jul 2026). Why AI notetakers fail to drive action.

TechCrunch (Jun 25, 2026). Notion Mail shutdown; Enterprisedna (Jun 2026): more than half of users never opened the inbox.

Firstup (Jun 2024): 57% see no intranet purpose. — Coveo via Nexinite (Aug 2025): up to 90% fail to deliver value.

dev.to (Nov 14, 2025). Why developers hate Jira. — Breeze.pm (Oct 2025). — ClickUp (2026) and Everhour (Sep 2025) on Linear's limits.

LeanLaw: \$25K-\$100K per year unbilled. — LawBillity (Aug 2025). — ABA Law Technology (Mar 2025) on AI time capture.

Reddit r/sysadmin (Dec 2025). — Medium (Jul 2026). The enterprise software delusion. — UX Matters (Feb 2025): ~70% transformation failure from adoption. — BCG (Oct 2025): up to 60% manual-workload reduction. — Turing Post on Glean's \$4.6B valuation.

Frontier gaps (Track R5)

dev.to (Jun 2026). A practical framework for testing non-deterministic AI: 362 documented incidents in 2025 (from 233 in 2024); hallucination rates 22-94% across 26 models.

unmeshed.io and truefoundry.com (Jun 2026): enterprise LLM API spend \$3.5B (late 2024) to \$8.4B (mid 2025). — groundcover.com on post-PoC abandonment.

arXiv (May 2026). Portable Agent Memory: A Protocol for Provenance — Mem0/Zep cloud memory as vendor lock-in.

intercypse.com (Aug 2026): 46% active distrust, 33% trust. — Dave Farley (Oct 2025), citing external research: 70% trust output they cannot fully explain.

PR Newswire (Jul 28, 2025): 27% of AECO professionals on Excel/PDFs. — premiercs.com (2026): 85% estimating in Excel. — ingenious.build on Excel persistence.

GoodShip (May 2026). TMS versus broker intelligence. — vektortms.com (Mar 2025). — aljex.com (Sep 2024) on legacy TMS UX.